

Benchmarking Off-the-Shelf Vision–Language Models on EEG Waveform Images

Technical and architectural state of the project

Luka • lukaj2501@gmail.com • 17 August 2026

EEG recordings from six sub-corpora of the Temple University EEG Corpus are rendered as multi-channel waveform plots and shown to 35 off-the-shelf vision–language models (VLMs), which are asked to classify them with no EEG-specific training. Everything runs through Ollama as Slurm array jobs on Narval. A second track fine-tunes a small model on written rationales produced by a larger one.

The corpus and the evaluation harness are complete. The full sweep has not been run yet, so this report describes the system and the reasoning behind it. Section 6 lists what is finished and what is still missing.

76,334	35 × 6	210	14,850
rendered windows	models × corpora	array tasks	windows scored

1 What the project is testing

The question is how well a general-purpose vision model can read an EEG when it is presented the way a clinician sees it, as a stacked plot of every channel against time. None of these models were trained on EEG, so the measurement is how far general multimodal capability transfers to an image type well outside the training distribution.

The framing needs to stay precise. Rendering a 40-channel signal as a picture discards fine morphology that a model reading the raw 1-D signal keeps, so a specialised EEG classifier should be expected to win. The useful comparison is not against that classifier but against what a model scores by ignoring the image entirely, which is why a constant-predictor baseline accompanies every result (Section 5.5).

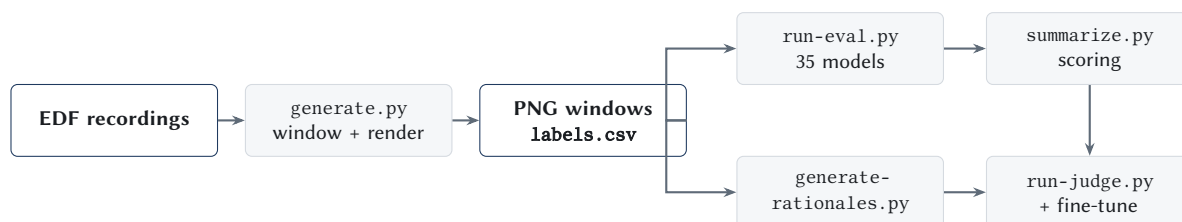


Figure 1: Stage layout. White boxes are durable artifacts on disk, grey boxes are stages. Every long-running stage resumes from partial progress, because these run as array jobs on a shared cluster where a queue can cost days.

2 Cluster environment

All GPU work runs on Narval (Digital Research Alliance of Canada, account def-milad777), whose GPU nodes carry A100-40 GB cards and support MIG. MIG (Multi-Instance GPU) partitions one physical card into hardware-isolated slices, each with its own memory and compute units, so several jobs share a card without contending for it. The project uses MIG heavily: a small model takes a 20 GB slice rather than occupying a whole 40 GB card.

That is not a stylistic preference. Early in development the project drew a resource-waste warning from the cluster operators for under-utilised full-GPU jobs, and most of the resource engineering described below is the response to it.

Work is submitted as Slurm *array jobs*: one submission expands into N indexed tasks that the scheduler places independently, so 210 model-and-dataset pairs queue as eleven submissions rather than 210.

2.1 Ollama

Inference is served by Ollama running inside an Apptainer container. Apptainer is the HPC container runtime (formerly Singularity) that runs unprivileged, so a user can bring their own userspace onto a compute node. It is required here because the native ollama binary is not on Narval's compute-node PATH; assuming it was there once left a rationale job spinning until its walltime expired. Two paths on scratch are the only manual setup step in the whole project: \$SCRATCH/ollama/ollama.sif (the container image, built once) and \$SCRATCH/ollama/models (the model store, shared by every job).

Every array task starts its own Ollama server. The port is derived from the job and task IDs as $20000 + (\text{job} + \text{task}) \bmod 40000$, so two tasks landing on one node cannot collide, and a shell trap kills the server on exit. The task waits up to 120 s for the server to answer and records the Ollama version alongside the run, since vision-model regressions land there often enough to matter (0.13.3 broke image input outright, ollama/ollama#13549).

Three settings are exported into the container, each read from the same place the Python client reads it so that server and client cannot disagree:

- **Requests in flight** (8 on a full A100, 4 on a MIG slice). Ollama batches concurrent requests, so this is what keeps the GPU busy instead of idling between images.
- **Context length** 8192. An image plus prompt is about 4,000 tokens, and the 4096 default truncated output mid-rationale.
- **Flash attention.** Flash attention is an attention kernel that computes attention in tiles kept in fast on-chip memory and never materialises the full $N \times N$ attention matrix, so memory use falls from quadratic to linear in sequence length and decoding is faster. Here it is enabled less for the speed than as a bug workaround: on the non-flash path Ollama never emits an end-of-sequence token on Qwen2.5-VL, and the reply runs to the token limit.

Compute nodes have no outbound network

Neither registry.ollama.ai nor huggingface.co is reachable from a compute node, so every model must be staged into \$SCRATCH/ollama/models from a login node *before* submission. This is a staging step, not a limitation to engineer around, but it has teeth: a pull fallback inside a compute-node job can only hang and then time out, reporting a network error rather than “model not staged”. That cost a whole judge array on 16 August. The sbatch wrappers now fail fast and name the missing model, and run-judge.py check verifies staging before anything is queued.

2.2 Hugging Face weights

Ollama serves quantised GGUF weights. GGUF is a single-file weight format packed for llama.cpp-style inference: fine for serving, but the tensors cannot be fed to a PyTorch LoRA fine-tune. LoRA (low-rank adaptation) freezes the base weights and trains a small pair of low-rank matrices alongside them, which needs the original full-precision checkpoint. hf-install.py therefore downloads complete Hugging Face snapshots (weights, processor, chat template) for the fine-tune track, keeping an explicit map from Ollama tag to HF checkpoint and writing a manifest.

It runs from a **login node**, because it needs outbound internet and compute nodes have none. Downloading all of it in one pass did not work: roughly forty repositories, several of them tens of gigabytes, with parallel connections, decompression and hashing throughout, exceeded what a shared login node permits one user, and the run died with an error partway through. The fix was not to make the script cheaper but to run it repeatedly over a handful of repositories at a time; hf-install.py checks each repository against the current remote revision and skips any already complete on disk, so a re-run costs only the remainder. The downloads were spread across roughly a week that way. Gated repositories need their licence accepted on Hugging Face first, and fail loudly until it is.

A Slurm wrapper for the script is in git history, but a compute node cannot reach Hugging Face, so the login-node path above is the working one. Both files are currently deleted from the working tree and need restoring

before the fine-tune track can proceed (Section 6).

3 Dataset generation

3.1 Download and layout

Source corpora are pulled from Temple’s ISIP server with `eval/scripts/download.sh`, a thin wrapper over `rsync -auvxl` so that an interrupted transfer resumes instead of restarting, which matters when TUSZ alone is roughly 85 GB of EDF. (EDF, European Data Format, is the standard clinical container for a multi-channel recording plus its per-channel metadata.) Each corpus unpacks into its own versioned directory inside its dataset folder, the layout the generators expect. `<CORPUS>` below is one of TUAB, TUAR, TUEP, TUEV, TUSL, TUSZ:

Path	Contents
<code>datasets/<CORPUS>/v*.*/</code>	raw EDF, never committed
<code>datasets/<CORPUS>/generate.py</code>	dataset-specific label logic and split
<code>datasets/<CORPUS>/labels.csv</code>	the manifest, committed to git
<code>datasets/<CORPUS>/{train,test}/</code>	rendered PNG windows, shipped by zip
<code>datasets/_render.py</code>	shared windowing and rendering engine
<code>datasets/generate-all.sh</code>	runs all six generators in parallel

Images and labels reach the cluster by different routes, which is worth knowing before changing either. `labels.csv` travels via git, the images as a 37.7 GB zip bundle. A label change is therefore a `git pull`; an image change means re-zipping and re-transferring everything.

3.2 Splitting and windowing

`generate-all.sh` runs the six `generate.py` scripts in parallel, each writing to its own directory. Each generator supplies four dataset-specific pieces (how to read the patient ID from a filename, how to name scans, how to split, and how to label a time range) and delegates the rest to the shared engine.

The split is 70/30 and grouped by *patient*. Patients are sorted, shuffled with a fixed seed, and assigned whole to test until about 30% of scans is reached, so every recording and window from one person lands on the same side. This is what stops a model being rewarded for recognising an individual’s EEG instead of the finding, and it is deterministic given the seed.

Recordings run from 4 to 50 minutes, which creates the central sampling problem. Rendering an entire recording into one image yields roughly one pixel per second and destroys the morphology the task depends on, so recordings are cut into non-overlapping 20-second windows. Rendering every window would then produce hundreds of thousands of images, so the two splits use different policies:

- **Train** takes a small mixed sample per recording (4 windows by default, seeded per recording): windows containing an annotated event are preferred, and the remainder is filled with background. The model sees both the finding and normal activity, cheaply.
- **Test** takes evenly-spaced background windows (capped at 10) plus *every* window containing an event. The test set therefore resembles the recording as a whole while still guaranteeing that rare findings are present.

An earlier version rendered only the first 20 seconds of each recording and missed events almost entirely, which is what motivated this design.

3.3 Rendering

Rendering is centralised in `datasets/_render.py` so that images are comparable across all six datasets. Each recording is read once with MNE (the standard Python library for EEG and MEG) using `read_raw_edf(preload=True)`. Eager preloading is required because TUEG files mix sampling rates across channels, and lazily loaded mixed-rate data produces edge artifacts at window boundaries. A 0.5–70 Hz bandpass and a 60 Hz notch (a narrow cut at the North American mains frequency) are applied once per recording, removing

drift and line hum so that recordings from different machines are comparable. Marker channels such as IBI and PHOTIC are skipped, since filtering a derived channel produces ringing.

Every channel gets its own subplot row, 30 to 41 rows per image, so traces never overlap. Images are 16×16 inches at 96 dpi, giving exactly 1536×1536 px. A VLM cuts an image into patches and charges each patch against its context, so resolution is a direct cost: this one is about 3,050 vision tokens. Legibility turned out to depend on font size and not on pixel count. 1536 and 2048 px produced identical read-back, while raising channel labels from 5 pt to 8 pt bold fixed it, and models now read back roughly 26 of 30 channel labels correctly.

Two further decisions shape what the models can see.

Amplitude is scaled per channel, to the 99th percentile of $|y|$ times 1.15. Getting there took three attempts: scaling to the median clipped badly (EEG has a crest factor around 12), and $8\times$ median overcorrected and buried the trace at roughly 12% of the row height, which reads as a flat line. Global scaling across channels was prototyped and rejected outright, because channels differ in amplitude by up to $25\times$ and the loud ones become unreadable solid blocks. The trade is that cross-channel amplitude comparison is lost, so asymmetry and attenuation cues are unavailable and a scale bar would be meaningless.

Nothing in the image identifies the answer. The EDF filename is never drawn on the plot. An early version used it as the title, and because TUEG filenames encode the class for several corpora (pled..., normal/...), models could read the answer straight off the image. Output files are named `<patient>_<scan>_<window>.png`, carrying no label and no dataset.

3.4 labels.csv and the assessed flag

Each dataset directory holds one manifest with a row per rendered window: the image path, the split, one boolean column per class, and an assessed flag. It is streamed as rendering proceeds, so a completed run leaves a complete manifest and a crashed one leaves an obviously partial file.

The assessed flag exists because of a label-quality problem found after the images already existed. The label function returned an empty set both when annotators had marked a window quiet and when they had never examined it, and generation wrote both as an all-false row. Scoring therefore graded never-inspected windows as though every class were absent, so they could only ever generate false positives and could never contribute to recall. This affected roughly 70% of TUSZ, 79% of TUEV and 75% of TUSL test windows.

Which meaning applies proved measurable per corpus, and the two cases get opposite treatment. TUSZ never mixes them in one annotation file (of 8,140 files, 7,043 are background-only and 1,097 seizure-only, and none holds both), and its seizure annotations are provably complete: the local copy reproduces the official AAREADME.txt event and duration statistics exactly. So absence of annotation genuinely is background there, and those windows were relabelled rather than dropped. TUEV, TUSL and TUAR annotate background explicitly, so an unannotated window means nobody looked, and those are excluded from scoring (assessed=false on 2,378 TUEV, 538 TUAR and 472 TUSL windows; zero elsewhere).

TUAR had a second, separate problem: its generator discarded the 3,122 background rows present in the source annotations, leaving a class that the prompt offered but that could never be correct, and a test set with no true negatives at all. They were recovered under a gate that refuses to write unless every pre-existing label first reproduces exactly from source.

Both corrections were pure labels.csv transforms with no image re-rendered, since a window's time span follows from its filename index and its recording from the generator's deterministic naming.

4 Ground-truth rationale generation

This stage shows a model the image *together with the correct label* and asks it to justify that label from visible evidence: which channel, where along the time axis, and what the waveform looks like. The prompt forbids describing the image format, defining terminology, or adding findings the label does not contain.

The teacher is gemma3:12b at temperature 0, meaning greedy decoding: the highest-probability token is taken at every step, so the same image always produces the same rationale. It was originally a Qwen2.5-VL model, which had to be replaced. In Ollama, Qwen2.5-VL falls into infinite single-token repetition on these plots,

Table 1: The corpus as it stands, re-derived from `labels.csv` and `eval/scripts/sample.py` on 17 August 2026. “Assessed” excludes windows no annotator examined; “sampled” is the subset actually scored (Section 5.2).

Corpus	Train	Test	Assessed	Sampled	Rec.	Pat.	Classes
TUAB	2,800	3,000	3,000	1,200	300	222	normal / abnormal
TUAR	863	3,915	3,664	3,655	94	67	6 artifacts + bckg
TUEP	6,868	8,610	8,610	1,084	285	113	epilepsy / none
TUEV	1,441	1,729	354	354	151	115	6 event types
TUSL	312	400	99	97	34	8	bckg / seiz / slow
TUSZ	21,207	25,189	25,189	8,460	2,443	200	bckg + 8 seizure types
Total	33,491	42,843	41,316	14,850			

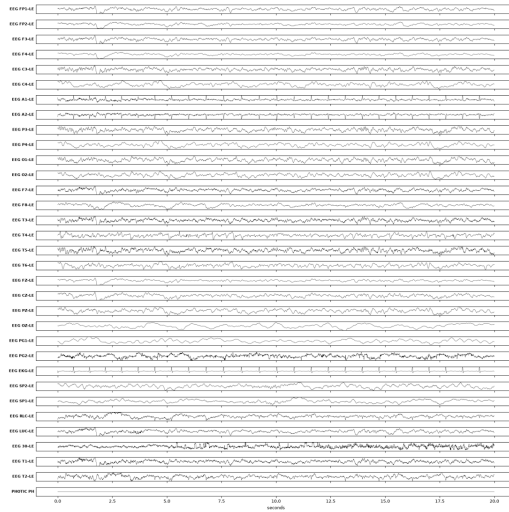


Figure 2: A representative model input: a 20-second window at 1536×1536 px with 31 channels on separate rows. Ground truth is `fnsz`, a focal non-specific seizure. Nothing in the image or its filename identifies the dataset, the class, or the source recording.

and the bug affects the 7b, 32b and 72b variants alike (`ollama/ollama#10767`), so a larger Qwen is not the fix. Ollama also exposes no repetition-loop detection (`#16179`) and neither the DRY nor the XTC sampler (`#7504`), so no sampler setting prevents it either. Changing model family was the only available lever.

One generator serves two consumers, selected by `--split`: `rationales.csv` for the fine-tuning targets, and `rationales-test.csv` for the reference rationales the judge compares against. The separation is load-bearing, since the test file is derived from held-out patients and training on it would void the patient split the whole benchmark depends on.

The engineering here is mostly about not corrupting the output. A degenerate reply, one too short or with too few distinct characters, is the signature of a wedged runner, and because decoding is greedy it repeats identically on every retry, so it passes every check except that one. Such a reply leaves the row blank so a later run retries it, and 20 consecutive degenerate replies abort the job rather than spending the rest of the allocation writing garbage into a fine-tuning set. Each run also rebuilds its target list from the current `labels.csv` while carrying over rationales already written, which fixed a crash where a changed manifest desynchronised the lookup mid-job.

5 Evaluation

5.1 Configuration

`eval/config.yml` is the single source of configuration, with four keys: `judge`, `settings`, `datasets` (the per-dataset prompts) and `models` (the roster with per-model resources). One invariant matters, because breaking it silently destroyed configuration once: every key the retry generator loads, it must also write back.

Each dataset’s prompt states that the image is a multi-channel EEG waveform plot with one channel per row, names the exact label vocabulary, requires the model to select every category present, and demands grounded per-item evidence. An earlier generation of prompts described the images as spectrograms, which is wrong and caused models to spend tokens noting the mismatch.

Predictions are never parsed from prose. `structure.py` builds a Pydantic model per dataset, one boolean per class plus a rationale string, and that schema is handed to the server as a JSON schema the decoder is constrained to satisfy. So a reply is either valid structured output or an explicit failure, never a paragraph to regex. It also bounds what the constant-predictor baseline is allowed to propose.

5.2 Test-set sampling

Compute scales with the number of windows, but statistical precision scales with the scoring unit, which is the recording (the patient for TUEP). Densely tiling background windows therefore consumes most of the compute while buying almost no precision.

`sample.py` caps abundant label signatures at 3 windows per recording, background-only windows at 2, binary-dataset windows at 4 per recording and recordings at 4 per patient, while keeping every window carrying an under-supported class in full. Selection is by even spacing with no RNG, so every model is evaluated on an identical set that is reproducible from `labels.csv` alone, with no sample file to go stale. After sampling, every dataset’s scoring-unit class support is identical to the full assessed set, so the reduction from 41,316 to 14,850 windows costs nothing that gets reported.

5.3 Dispatch and execution

`run-eval.py` buckets the 210 (model, dataset) tasks by their resource requirements into 11 groups, creates a run directory containing logs, results, a status file and a frozen copy of the config, submits one Slurm array per group, and records job and task IDs for later reconciliation. The roster allocates 21 models to a 20 GB MIG slice, 13 to a full A100, and one mixture-of-experts model to four cards. (A mixture-of-experts model activates only a fraction of its parameters per token but must still hold all of them resident, so it is cheap to run and expensive to place.)

Three sizing rules decide whether a job runs at all, and all three are computed rather than guessed:

- **VRAM:** $\text{weights} \times 1.15 + \text{KV cache} \times \text{requests in flight}$. Below that, Ollama silently offloads layers to CPU and runs orders of magnitude slower rather than failing.
- **RAM:** $1.6 \times \text{weights} + 8 \text{ GB}$, and never above the ~498 GB a whole Narval GPU node has. Retuning these requests took the roster total from 2,488 GB to 1,008 GB, the single largest lever on queue wait.
- **Walltime:** $2.5 \times$ an estimated TUSZ task, TUSZ being the largest corpus and therefore the bound for all six. These are estimates, not measurements.

Ten further models sit commented out in the roster with their reason recorded: three suspected Ollama tag aliases, five of the six -thinking variants (the reasoning ablation is run once, at 8b, against the matching instruct model), and two that do not fit the hardware. The clearest of those is `qwen3-v1:235b-a22b`, whose Q4 weights are about 142 GB against the 160 GB of VRAM on four A100-40 GB cards, leaving nothing for activations, the vision tower or the KV cache; its earlier 512 GB RAM request also exceeded the node total, so that job could never have been scheduled at all. It is excluded rather than deleted, and needs 80 GB-class cards.

Each task decodes its model and dataset from the array index, starts Ollama, restricts `labels.csv` to test patients, applies the sampling policy, and runs the images through a thread pool sized to the model’s in-flight setting. Three behaviours are chosen on purpose. A per-image failure is logged and the run continues, so one bad image cannot lose a task’s work. Retries back off only for transport errors, since a schema-validation failure is deterministic and pausing before repeating it just burns walltime. And completed image paths are read from both the current run and any previous run being resumed, so a Slurm requeue picks up where it stopped.

5.4 Retry, merge and monitoring

Long jobs on a shared cluster fail partially, so three tools exist to make that cheap to handle.

`status.py` reconciles the run’s status file against live `squeue` and `sacct` state and writes back outcomes for tasks that died without recording their own failure, such as one killed by the walltime. It had a bug worth recording: it asked `sacct` for `JobIDRaw`, which numbers array *elements* individually (556372) instead of `<array>_<task>` (555281_0), so a guard on the underscore skipped every finished element and the write-back never fired. Eleven tasks sat at running against an empty queue before it was found.

`create-retry-config.py` rebuilds the config to re-run only the pairs that failed and points `resume-from` at the run being retried. Finally, `merge-runs.py` unions the base run and the retry into one scoreable directory, and this step is not optional: `resume-from` only tells the retry which images to skip and never copies base rows forward, so neither directory is complete alone and scoring either one under-reports.

5.5 Scoring

`summarize.py` is the single scoring stage. Two things motivate its design. Accuracy on a background-dominated, window-tiled test set flatters a model that always answers background. And TUAB and TUEP are labelled at the recording level, so grading one normal-looking window against its recording’s label is not a fair unit.

The headline metric is therefore recording-level macro-F1: per-class F1 averaged with every class weighted equally, so a rare seizure type counts as much as background. A recording’s windows are aggregated into one prediction, by majority vote for the binary datasets and by any-positive detection for the multi-label ones, and a class needs at least 20 positive recordings to enter the average. Three guards accompany it:

- A 95% bootstrap CI that resamples whole recordings rather than windows. Windows within a recording share a patient, montage and session, so resampling them individually would treat correlated observations as independent and understate the interval.
- A constant-predictor baseline: the best macro-F1 an image-blind constant answer could achieve on the same data, restricted to answers the schema can emit. That restriction is not cosmetic, since TUAB and TUEP resolve to a single boolean, and allowing “predict everything” would put TUAB’s floor at 0.666 instead of 0.346 and screen out every model.
- A degeneracy flag for any model giving the same answer on 95% or more of windows, and a descriptive-only marker for datasets with fewer than 20 patients (TUSL has 8).

Scoring used to be two scripts, and the split was a trap worth naming. The rigorous metrics were computed by one script that only printed them, while the durable artifacts (`summary.csv`, `rank.csv`, `rank.png`) were written by the other from window-level accuracy. The misleading view was the one that persisted to disk and travelled into slides, and `rank.csv` ranked a model that only ever answered “background” near the top. They are now one script, verified equivalent before the old one was deleted, and the ranking sorts on recording-level macro-F1.

Outputs are `summary.csv` (one row per model and dataset, carrying the metrics together with `baseline_macro_f1`, `clears_baseline`, `degenerate`, `few_patients` and the task status, so a number cannot be read without its caveat), `classes.csv`, `rank.csv` and a ranking chart. The sweep runs in two stages: all 35 models are screened on a stratified probe first, and a model is promoted to the full test set only when its macro-F1 CI lower bound clears the printed baseline. Most general-purpose VLMs will probably sit near chance, and showing that does not require the full test set.

5.6 Rationale agreement

A final stage grades *why* a model answered as it did, comparing each model’s rationale against the reference rationale for the same window. The judge is `mistral-small13.2:24b`, used text-only so it never sees the image.

It was picked from the 35 models already staged on scratch, since a compute node cannot pull a new one, under five constraints: not the `gemma3:12b` teacher and not the `gemma` family at all, since `gemma` writes

every reference; not Qwen2.5-VL, for the repetition loop above; not a reasoning model, because token cost has to stay predictable across roughly 520,000 pairs; large enough to emit the schema reliably, which 8B-class models do not; and from the smallest capable family, so that as few graded rows as possible share the judge’s lineage. The cost of picking from the roster is that the judge is one of the models it grades, which is stated as a limitation in Section 7 rather than papered over.

The verdict is two booleans, same conclusion and same evidence, which separate a model that named the right finding off the wrong feature from one that named the wrong finding. A deterministic 5% of pairs are judged again against a *different* recording’s reference, and that false-agreement rate is published as a control floor. This targets the failure mode that makes LLM judges useless here, where a generic phrase such as “rhythmic activity across several channels” matches every reference. The headline figure is agreement restricted to correctly-labelled windows, separating right-for-the-right-reason from right-by-luck.

The stage is built but blocked: the teacher pass so far covers train patients while the benchmark runs on test patients, so the two sets are disjoint by construction and joining them returns zero rows. The `--split test` reference pass has to run first, and `run-judge.py check` verifies the join locally before any GPU time is spent.

6 Status and remaining work

Component	Status	Notes
Corpus generation	COMPLETE	76,334 images, patient-split, matching <code>labels.csv</code> .
Label corrections	COMPLETE	assessed in all six; TUAR background recovered.
Test-set sampling	COMPLETE	14,850 windows, reported class support unchanged.
Evaluation harness	COMPLETE	210 tasks; resume, retry, merge and status in place.
Scoring	COMPLETE	Recording-level macro-F1, bootstrap CI, baseline, degeneracy.
Rationale gen. (train)	PARTIAL	Runs on the cluster; local copy is stale.
Rationale gen. (test)	NOT STARTED	Hard prerequisite for the judge stage.
Rationale agreement	BLOCKED	Built; waiting only on the reference pass above.
Fine-tune	PARTIAL	Trainer written; no data converter, no Slurm wrapper.

Critical path. Re-run the train rationale pass so it matches the current `labels.csv`; submit the test-split reference pass, which is resumable and can run concurrently with the sweep; run the sweep and merge retries before scoring; then verify judge coverage before spending judge GPU time.

Three gaps in the working tree, all worth resolving before publication:

1. `datasets/_render.py` and `qc.py` are gitignored and are absent from the working tree. Since every `generate.py` imports `_render`, dataset generation cannot currently be re-run, and the exact code that produced the corpus is not archived anywhere recoverable. Nothing in the current workflow needs it, because label changes are always applied as manifest transforms, but this is a genuine reproducibility risk.
2. `hf-install.py` and its Slurm wrapper are committed but deleted from the working tree, blocking the fine-tune track.
3. The fine-tune trainer reads `sft_train.jsonl` while the rationale generator writes `rationales.csv`, and no converter exists between them.

7 Known limitations

- **Some classes are too rare to score.** Source recordings per class: `mysz` 2, `spsz` 4, `elpp` 4, `tnsz` 10 and `absz` 19, against `fnsz` at 655. A patient-level split often places such a class entirely on one side. They stay in the task, so the prompt is not quietly made easier, but are held out of headline metrics by a support threshold and reported with their counts. Exclusion is by support and never by performance.
- **The judge is one of the 35 models it grades.** Compute nodes cannot pull, so the judge had to come from

the staged roster, and every member of that roster is being benchmarked. `mistral-small13.2:24b`'s own row therefore grades its own prose and should be treated as non-comparable, and `mistral-small13.1:24b` shares its family. Staging an outside judge such as `qwen2.5:14b-instruct` from a login node closes this and costs one array job and no new teacher GPU time.

- **The judge is unvalidated against human labels**, so its output should be quoted as agreement as judged by one model. What is defensible is the ordering of models and each model's gap to its own control floor. Hand-labelling around 100 pairs, and re-judging a subsample with a second judge for Cohen's κ (a chance-corrected agreement statistic), are both cheap and both still undone.
- **Reference rationales inherit the teacher's ceiling**. They are `gemma3:12b` reading the plot, with no neurologist involved, so the stage measures agreement with the teacher and a model that disagrees may be correct.
- **Binary datasets are weakly supervised in training**. Every TUAB and TUEP training window inherits its recording's label, so some windows marked abnormal look normal. Recording-level aggregation handles this at scoring.
- **Throughput and walltimes are unmeasured**, coming from a weights-size model. Every task resumes, so an underestimate costs a requeue and no lost work.
- **`qwen2.5v1:3b` cannot reach full coverage** and needs a decision before reporting. It fails with an empty Ollama stream, largely image-deterministically: on retry, previously failed images failed again at 72–82% against 42% on first exposure, so each pass recovers about a fifth of the remainder and never converges.
- **Everything rests on one split seed**. A sensitivity check and multiple-comparison control across 35 models and 6 datasets are outstanding. Pooling corpora would also need a global patient split, since all six share an anonymisation scheme and a subject may appear in more than one.
- **Results are only reproducible if the environment is pinned**. The Ollama version and each model's weight digest both change outputs (quantisation is part of the model), as do matplotlib and font versions, which change the rendered image. The generation seed is already fixed.

A Terms used in this report

Term	Meaning here
Apptainer	HPC container runtime (formerly Singularity); runs unprivileged, so a user can bring their own userspace onto a compute node.
Bootstrap CI	Confidence interval from resampling the data with replacement. This one resamples whole recordings, not windows.
EDF	European Data Format, the standard clinical file for a multi-channel recording plus its channel metadata.
Flash attention	Attention kernel that works in on-chip tiles instead of materialising the full attention matrix; linear rather than quadratic memory.
GGUF	Single-file quantised weight format used by Ollama. Good for serving, unusable for a PyTorch fine-tune.
KV cache	Per-request memory holding attention keys and values for tokens already generated. Scales with context length and concurrency.
LoRA	Low-rank adaptation: freeze the base weights, train a small pair of low-rank matrices alongside them.
Macro-F1	Per-class F1 averaged with equal weight per class, so a rare class counts as much as a common one.
MIG	Multi-Instance GPU: one A100 partitioned into hardware-isolated slices with their own memory and compute.
MNE	The standard Python library for EEG and MEG signal handling.
MoE	Mixture of experts: only a fraction of parameters is active per token, but all must be resident in VRAM.
Montage	The electrode referencing scheme a recording is written in.
Slurm array job	One submission that expands into N independently scheduled indexed tasks.
Structured output	Constraining the decoder to a JSON schema, so a reply is valid or an explicit failure, never prose to parse.
Support	Number of positive examples of a class at the scoring unit. Below 20 recordings a class is reported but excluded from macro-F1.
Vision tokens	An image is cut into patches and each patch consumes context, so resolution is a direct inference cost.
Window	A non-overlapping 20-second slice of a recording; one window renders to one image and is one model input.